



Vibecode PHX
Workshop One

Oct 4 • Phoenix • 9AM–5PM

Vibecode PHX

Workshop One / Intro Into Vibecoding



What Is Vibecoding?

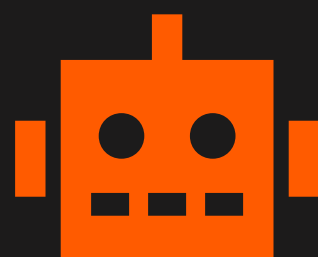
Vibe coding is intent-driven development.

You describe the behavior and constraints; the AI drafts the code; you run, review, and iterate. It removes friction between concept and execution so nonengineers and engineers alike can ship working software fast.



From idea to execution, fast

Removes friction, turning concepts into working prototypes quickly



Intent → Implementation loop

You describe behavior & constraints → AI drafts → you run, review, and iterate.



Democratizing software creation

Anyone (not just engineers) can now build apps and tools with AI tools and agents.

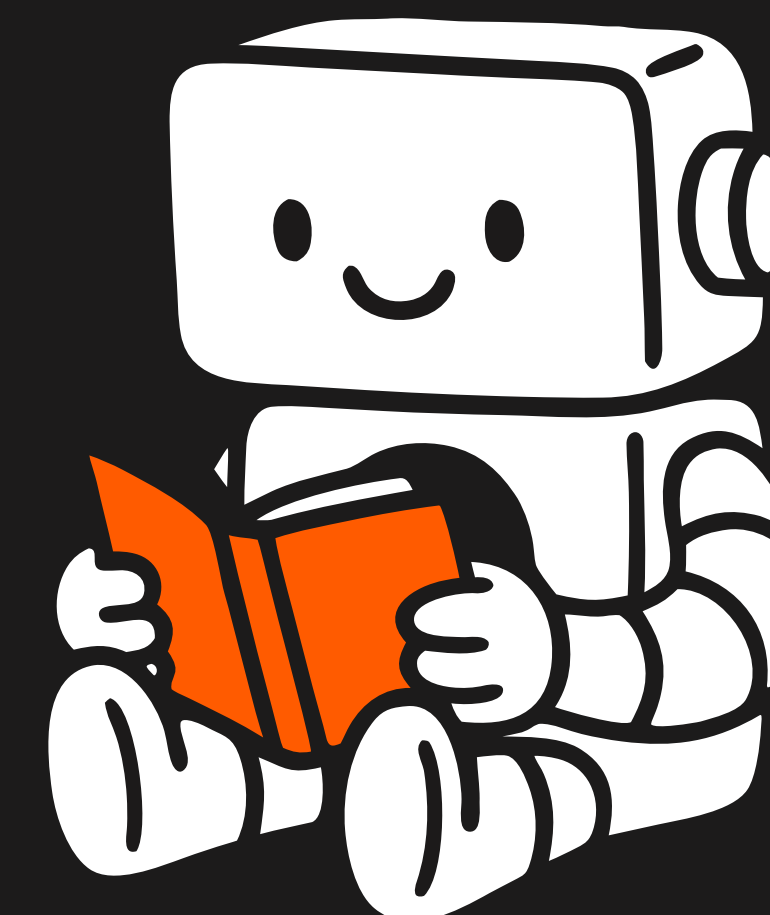


Why This Matters

Less drag,
more shipping



Everyone Can
Turn Ideas Into
Software.



In today's world, where less time is wasted on distractions and more focus is on shipping, transforming your ideas into code has never been easier. You'll walk away with a working app, a solid foundation for future enhancements, and a streamlined process that shields you from chaotic, fragile code.

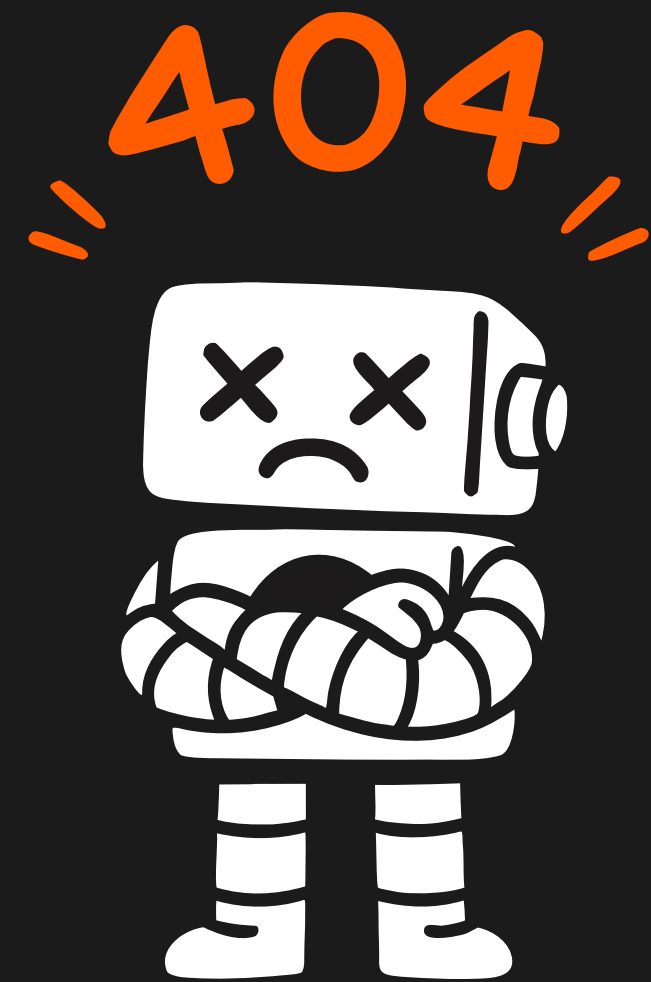


When you ask for too much from AI, it often produces unsatisfactory results. You try to fix the issues, but then something else goes wrong, leading to frustration and a loss of motivation. This cycle, known as the Doom Loop, arises from unrealistic expectations, lack of organization, and insufficient guidelines. Today, we aim to break this cycle.



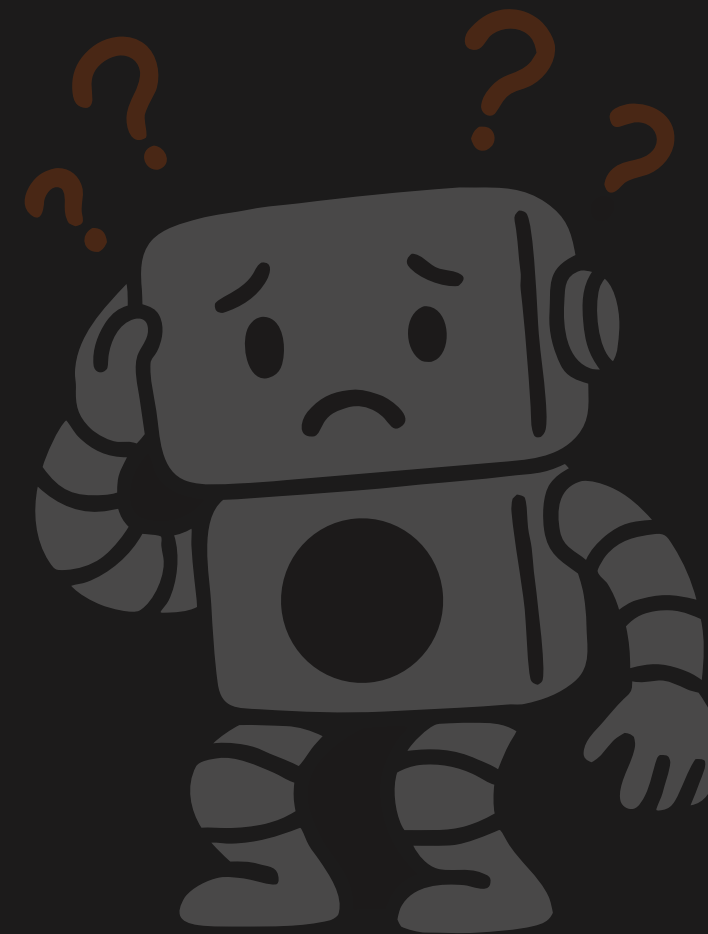


Doom Loops Are Usually Caused By The Following



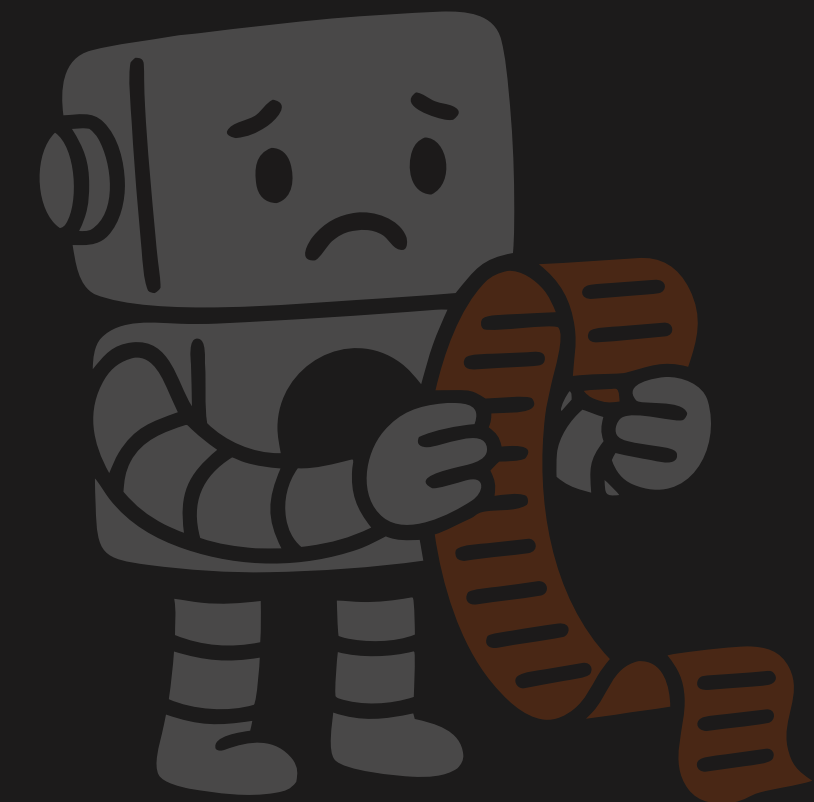
Overscope & Drift

Vague, oversized prompts and no plan produce sprawling code, schema drift, and unreproducible changes.



Quality & Security

Hallucinated/obsolete APIs, wrong versions, and missing validation yield brittle builds and bugs.



Data Explosion

When the same data appears in multiple places, updates become painful and reports show conflicting information.

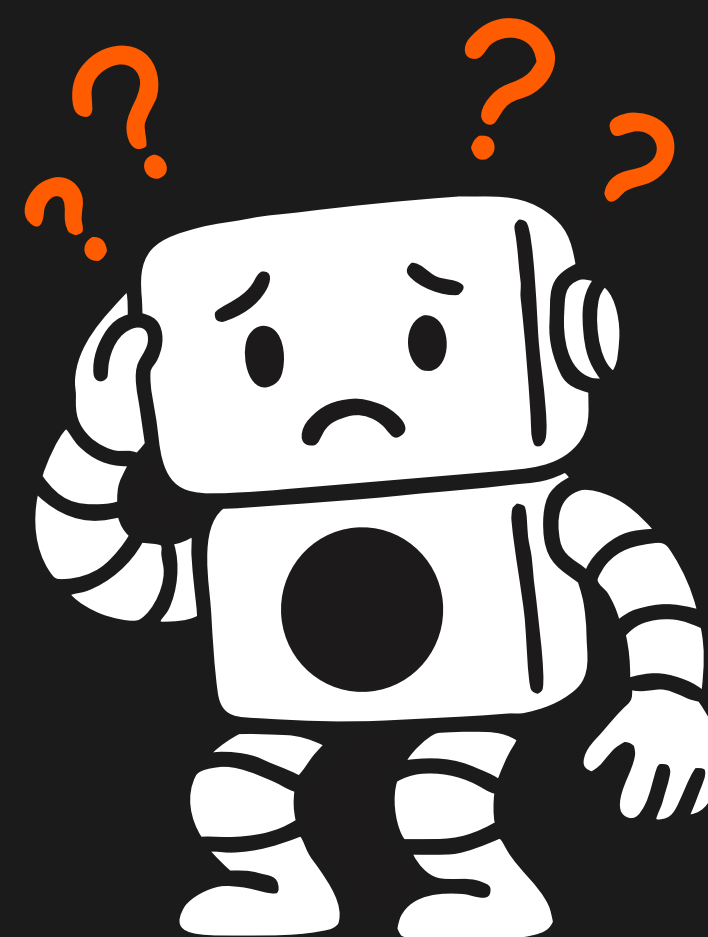


Doom Loops Are Usually Caused By The Following



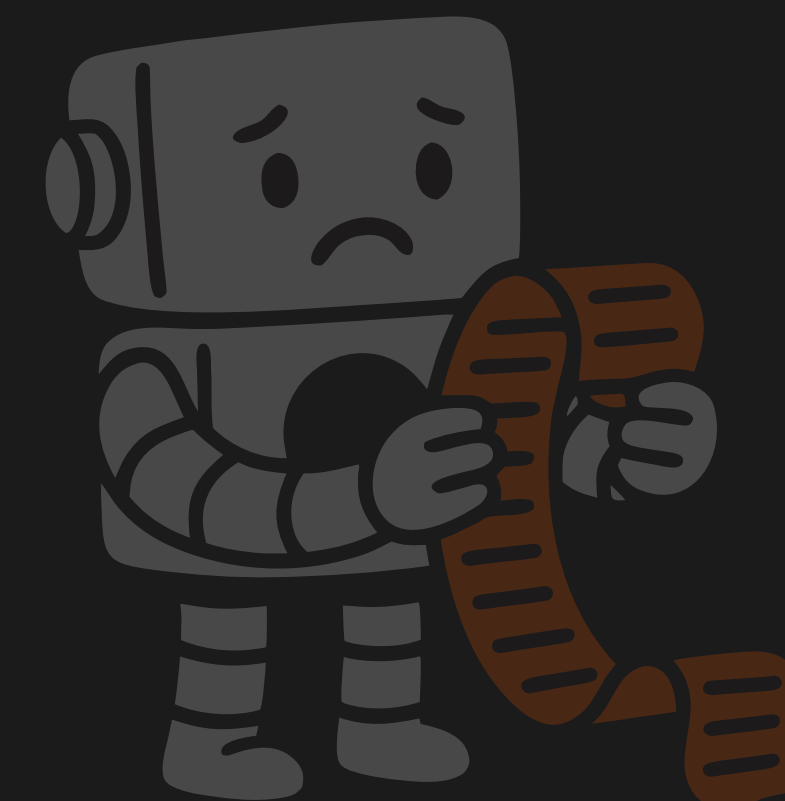
Overscope & Drift

Vague, oversized prompts and no plan produce sprawling code, schema drift, and unreproducible changes.



Quality & Security

Hallucinated/obsolete APIs, wrong versions, and missing validation yield brittle builds and bugs.



Data Explosion

When the same data appears in multiple places, updates become painful and reports show conflicting information.

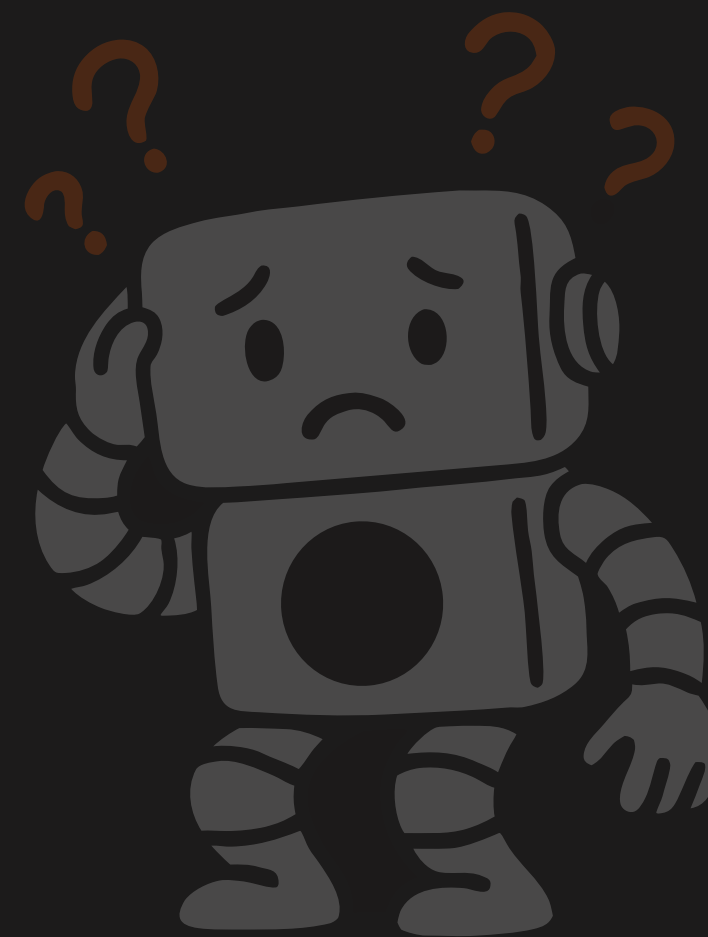


Doom Loops Are Usually Caused By The Following



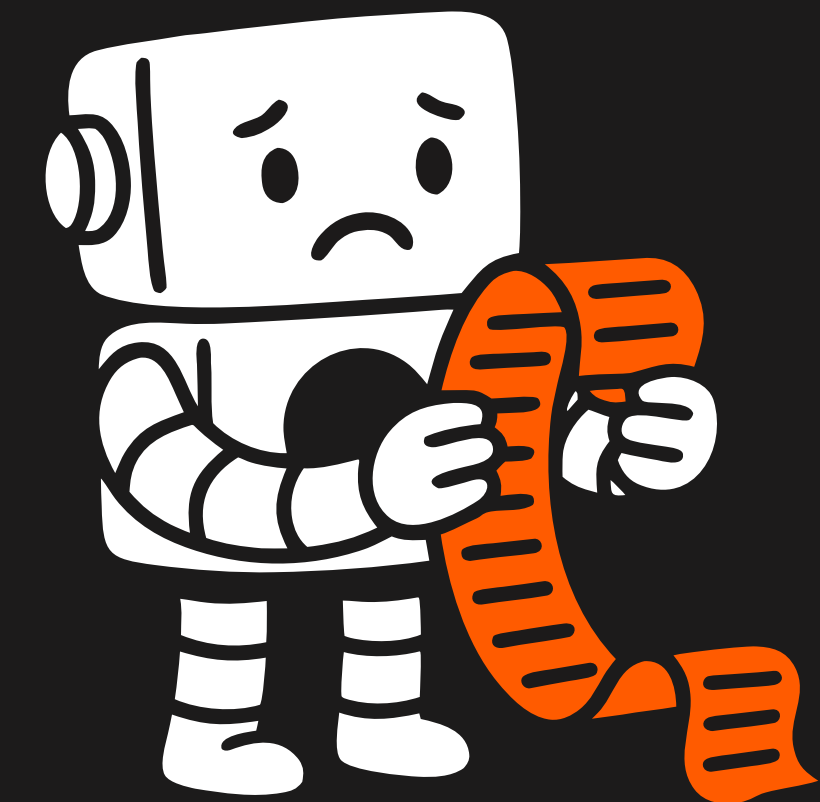
Overscope & Drift

Vague, oversized prompts and no plan produce sprawling code, schema drift, and unreproducible changes.



Quality & Security

Hallucinated/obsolete APIs, wrong versions, and missing validation yield brittle builds and bugs.



Data Explosion

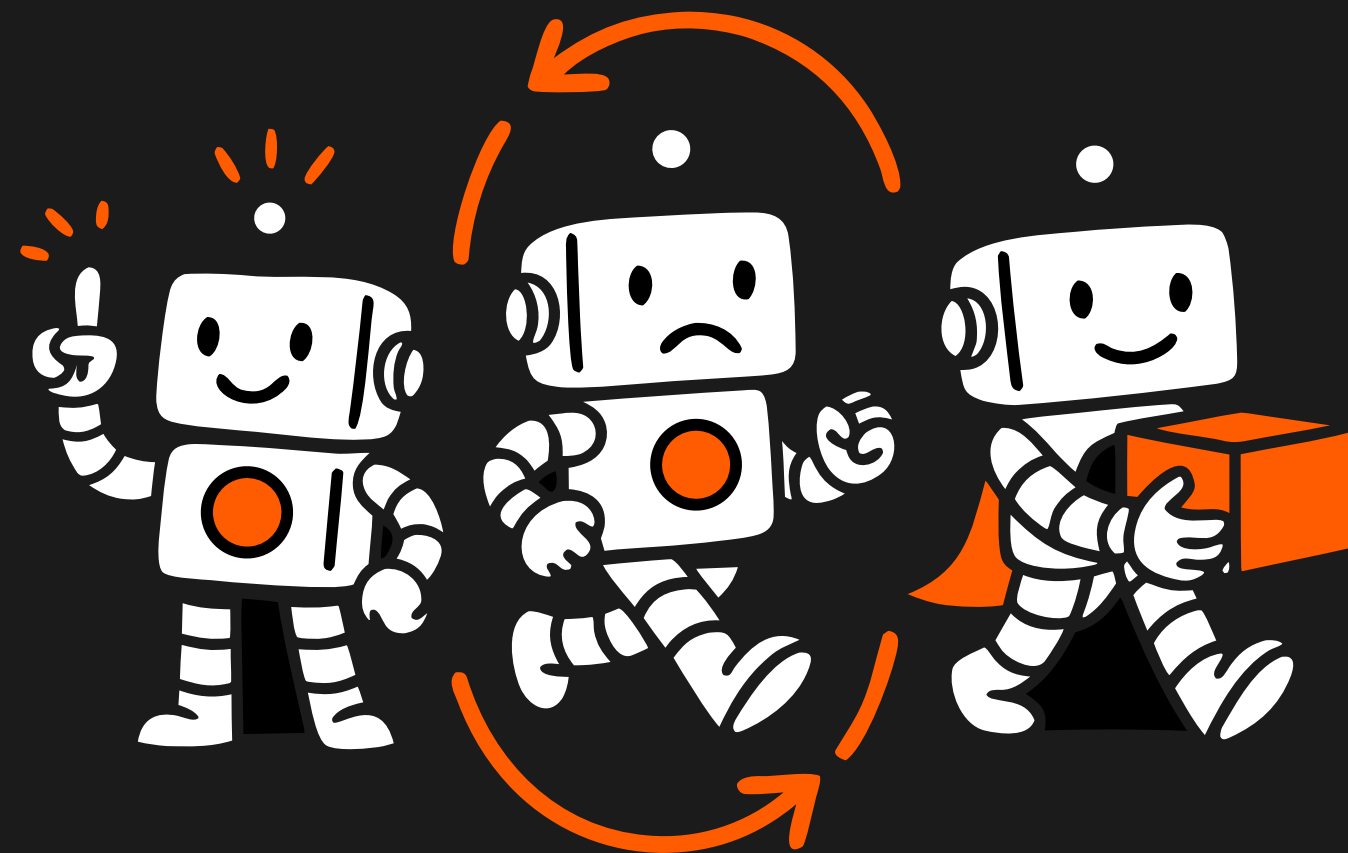
When the same data appears in multiple places, updates become painful and reports show conflicting information.



Vibecode PHX
Workshop One

Oct 4 • Phoenix • 9AM-5PM

How To Break The Doom Loop



The Plan, Loop, Ship: Method

Plan What You Want

Address the five fundamental questions: Who is the target audience? What is the expected outcome for today? How will we measure success? What steps or screens are necessary to achieve this? What potential issues might arise

Build Features in short loops

When approaching the development process, it's advisable not to tackle the entire application in one go. Instead, break it down into individual features, build and test each one, and then proceed with the next steps.

Ship early, learn fast

To improve usability, start by sharing a link that can be easily distributed. Observe a user interacting with it, paying attention to any points of confusion or difficulty. Based on this feedback, prioritize achieving the next incremental improvement in the user experience.



How To Plan Better With AI

Break Complex Problems into Single-Tasks

Instead of asking AI to "build an entire app," decompose your project into discrete, verifiable tasks.

Provide Context Before Code

Start your planning sessions by having AI help you write requirements, not implementation.

Use AI as a Reviewer, Not Just a Generator

After planning something yourself, ask AI to critique it: "What's missing from this database design?" or "What errors might this API structure cause?"



How To Build Features Better With AI

Slice to a Single Outcome

Define success as one deployed feature that users can actually interact with, not a collection of half-built components.

Set Hard Boundaries

Limit each loop into time or features, with a deployable result required at the end. If you can't finish in that time, you've scoped too large—break it down further.

Test .. Ship .. Repeat

Don't wait for "feature complete" to get feedback. Deploy your single feature and have someone actually use it within the same day.



Ship Often, Build Fast

Make Previewing Frictionless

Deploy to a real URL immediately, not localhost or a staging server. A simple Vercel preview link that anyone can open on their phone beats a perfect local setup

Watch, Don't Ask

Using your app can uncover valuable insights from users' confusion, false starts, and workarounds. Pay attention to where they click when nothing occurs, what actions they attempt that aren't feasible.

Fix the Biggest Frustration First

After each session, identify the main friction point and fix it before the next problem. Avoid rebuilding everything or adding unrequested features.



The Vibecoding Stack



CREATE

Turn ideas into starter screens in minutes. Speak or sketch; AI drafts the first pass.



BUILD

Describe what it should do; AI writes the code. You test, tweak, and keep each change small.



CONNECT

Plug in email, payments, calendars, or data sources. Guided steps—no deep setup.



STORE

Save sign-ups and app data safely in one place. No messy spreadsheets; easy to find and trust.



SHIP

Publish a live link today.
Get feedback fast, then repeat.



The Vibecoding Stack



CREATE

Turn ideas into starter screens in minutes. Speak or sketch; AI drafts the first pass.



BUILD

Describe what it should do; AI writes the code. You test, tweak, and keep each change small.



CONNECT

Plug in email, payments, calendars, or data sources. Guided steps—no deep setup.



STORE

Save sign-ups and app data safely in one place. No messy spreadsheets; easy to find and trust.



SHIP

Publish a live link today.
Get feedback fast, then repeat.



The Vibecoding Stack



CREATE

Turn ideas into starter screens in minutes. Speak or sketch; AI drafts the first pass.



BUILD

Describe what it should do; AI writes the code. You test, tweak, and keep each change small.



CONNECT

Plug in email, payments, calendars, or data sources. Guided steps—no deep setup.



STORE

Save sign-ups and app data safely in one place. No messy spreadsheets; easy to find and trust.



SHIP

Publish a live link today.
Get feedback fast, then repeat.



The Vibecoding Stack



CREATE

Turn ideas into starter screens in minutes. Speak or sketch; AI drafts the first pass.



BUILD

Describe what it should do; AI writes the code. You test, tweak, and keep each change small.



CONNECT

Plug in email, payments, calendars, or data sources. Guided steps—no deep setup.



STORE

Save sign-ups and app data safely in one place. No messy spreadsheets; easy to find and trust.



SHIP

Publish a live link today.
Get feedback fast, then repeat.



The Vibecoding Stack



CREATE

Turn ideas into starter screens in minutes. Speak or sketch; AI drafts the first pass.



BUILD

Describe what it should do; AI writes the code. You test, tweak, and keep each change small.



CONNECT

Plug in email, payments, calendars, or data sources. Guided steps—no deep setup.



STORE

Save sign-ups and app data safely in one place. No messy spreadsheets; easy to find and trust.



SHIP

Publish a live link today.
Get feedback fast, then repeat.

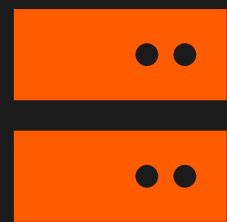


Anatomy of an Application



Editor

The environment for writing, debugging, and testing code. Includes IDEs and version control.



Backend

Handles business logic, authentication, APIs, and data processing. Manages security and core functions.



Frontend

UI layer presents data and interactions. Manages rendering, state, and user experience.



Database

Organizes and manages application data. Ensures integrity and efficient retrieval.



Connection

Protocols and APIs connect components, managing data flow.



Editor

Every build begins with your toolkit. Editors like Cursor, VS Code, or Windsurf give you a workspace where you can write and refine code with AI.

TOOLS OF THE TRADE



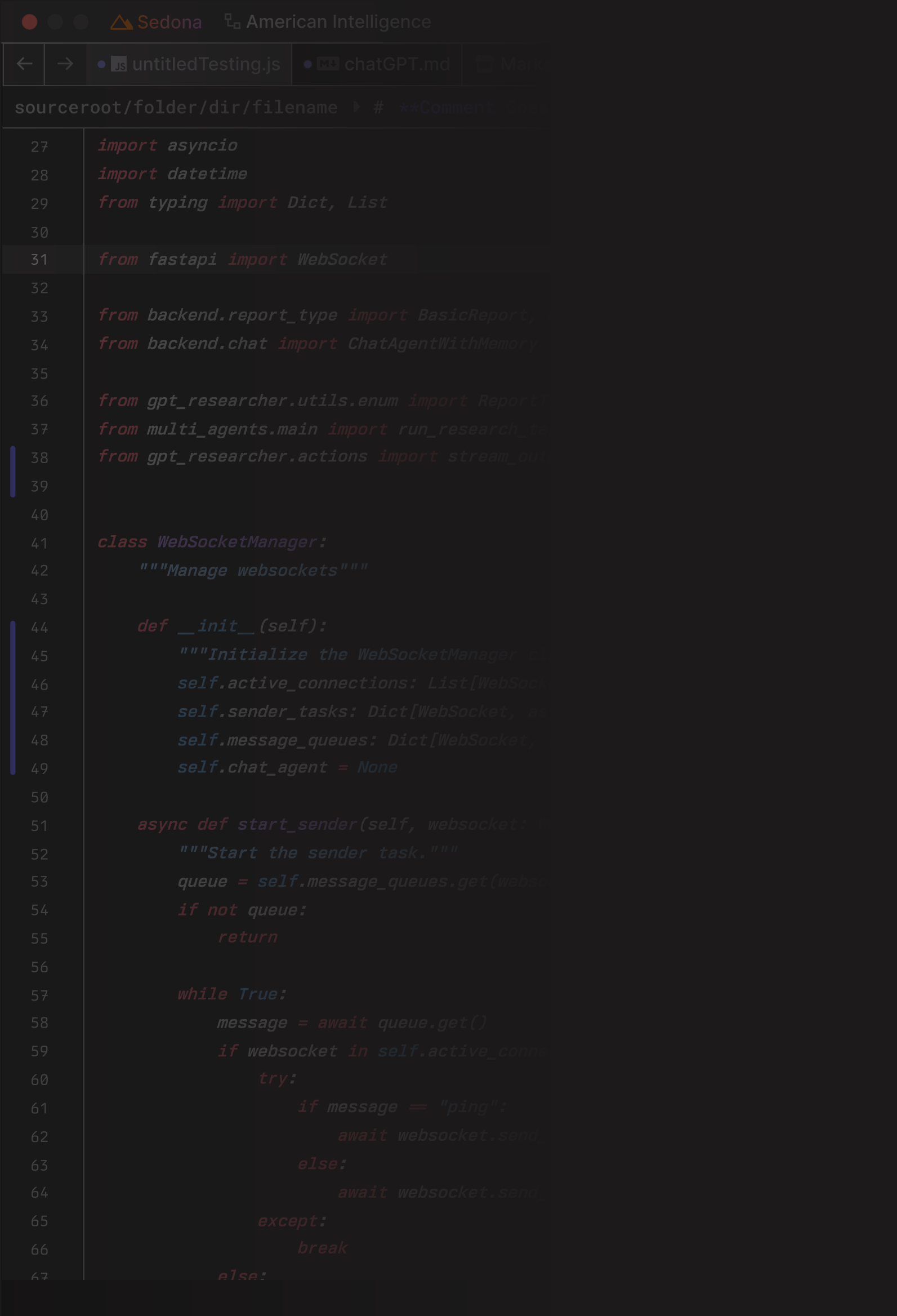
Anatomy Of An Editor

CODE DEVELOPMENT ENVIRONMENT

Every build begins with your toolkit. Modern editors transform raw ideas into polished code through intelligent autocomplete, real-time error detection, and integrated debugging. Built-in Git support tracks every change while extensions customize your perfect workflow.

QUICK POINTS

- 1. Smart code completion
- 2. Live error detection
- 3. Integrated debugging
- 4. Version control built-in





What Is Version Control Why Is Github Important?

Save Your Work

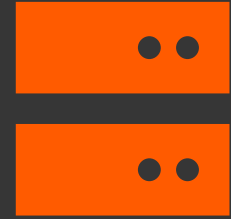
Branch Your Code

Fork Others Code

Remote Work



Are Certain Code Editors Better Than Others?



Backend

Server-side manages business operations, authentication, APIs, and data processing. Also handles security and core functions.

TOOLS OF THE TRADE



Anatomy Of An Backend

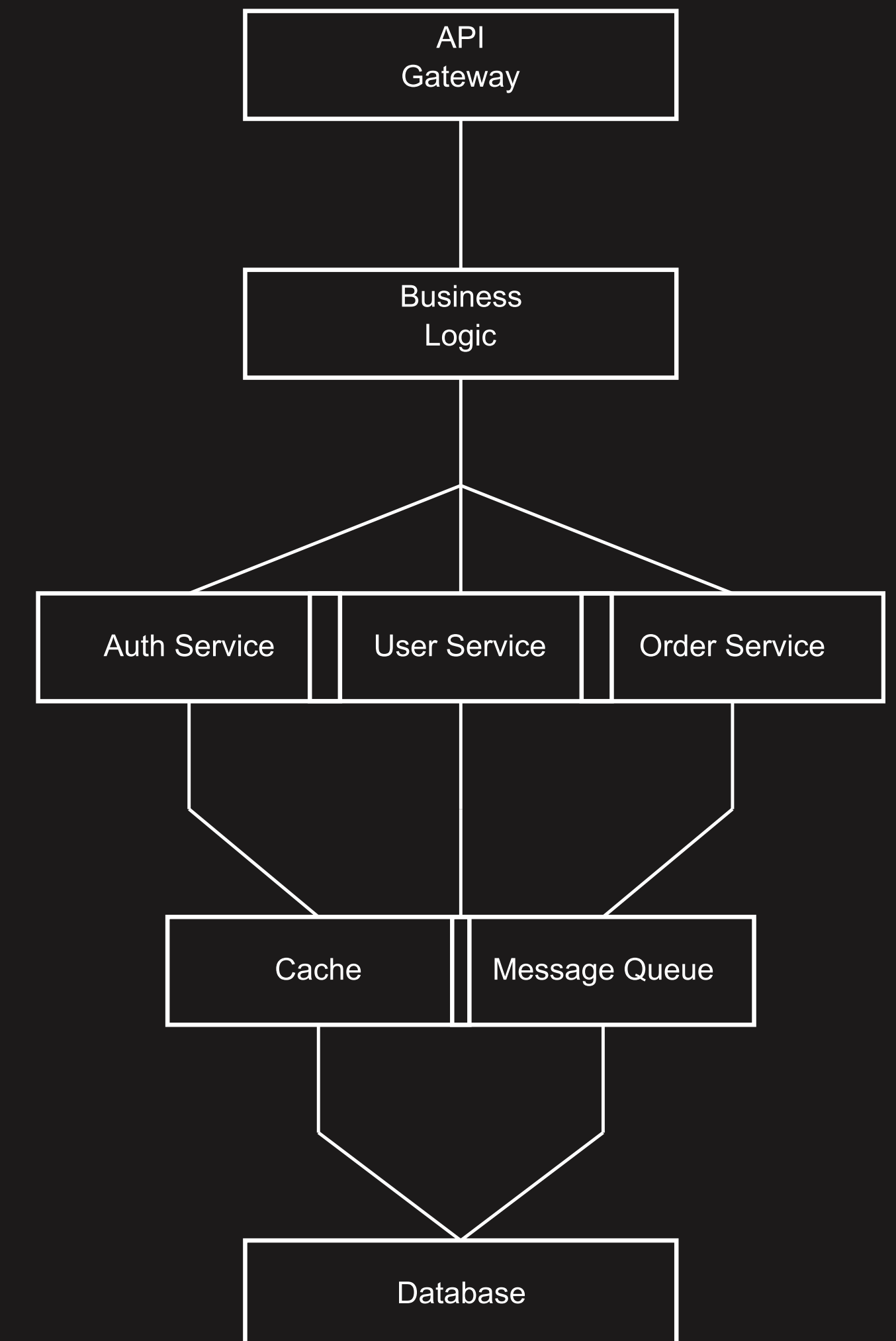
THE BRAIN OF YOUR APP

The backend orchestrates everything users don't see. It validates requests, enforces business rules, manages authentication, and coordinates data flow. APIs serve as gateways while microservices divide complex operations into manageable pieces.

QUICK POINTS

1. API endpoints & routing
2. Authentication & security
3. Business logic processing
4. Database communication

Backend *Architecture*





Your Backend Is Handles All The Behind The Scenes Logic

User Auth

How Data Flows

Application Security

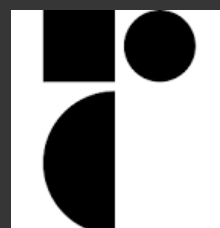
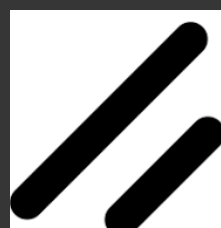
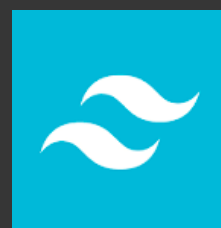
Application Logic



Frontend

User interface layer presenting data and enabling interactions. Handles rendering, state management, and user experience design.

TOOLS OF THE TRADE



Anatomy Of An Frontend

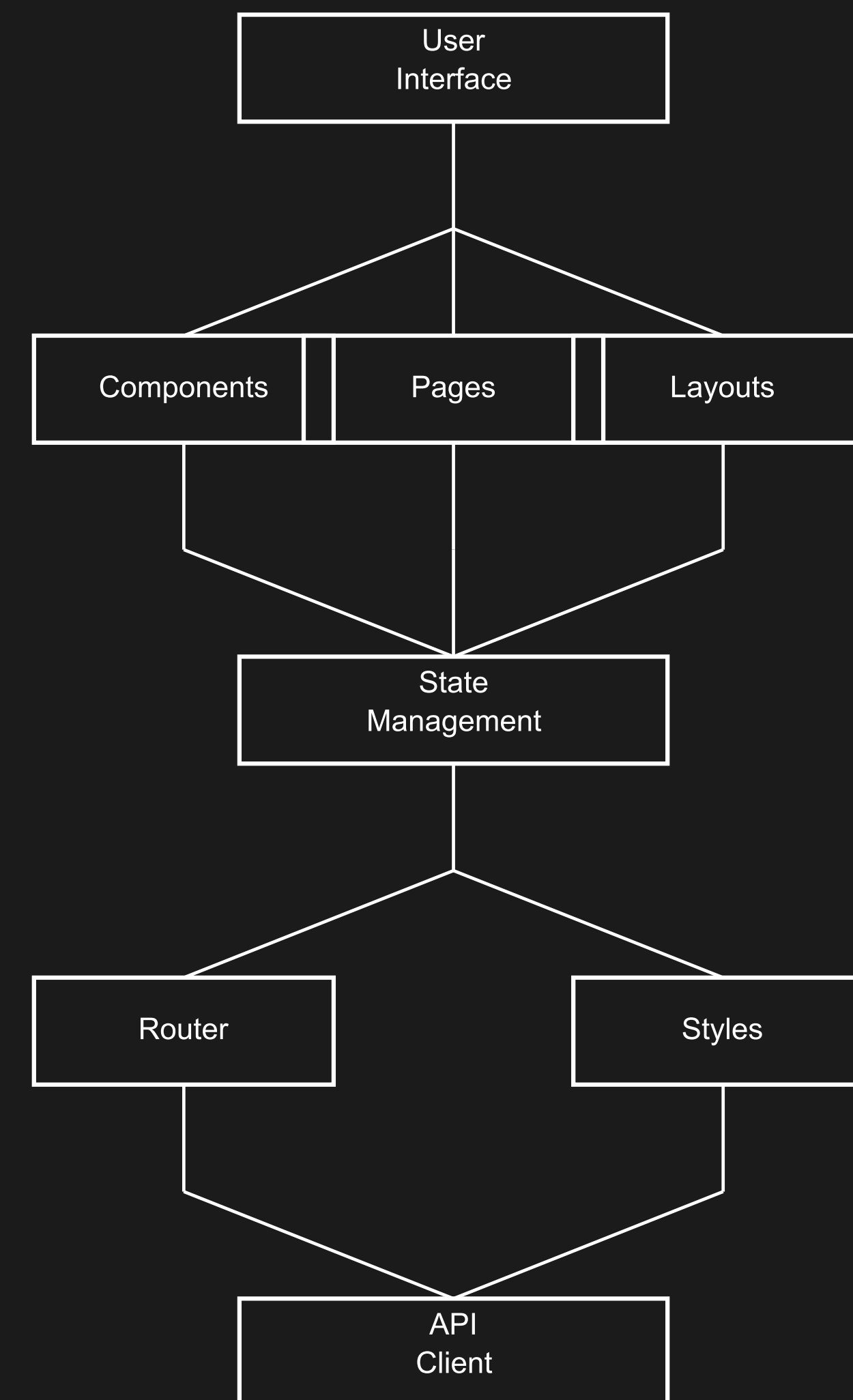
WHERE USERS MEET YOUR APP

The frontend transforms data into experiences. Components build responsive interfaces that adapt to any screen. State management keeps everything synchronized while modern frameworks ensure smooth interactions and lightning-fast updates.

QUICK POINTS

1. Smart code completion
2. Live error detection
3. Integrated debugging
4. Version control built-in

Frontend *Architecture*





How Do Frontend And Backend Actually Talk?

HTTP Requests
Like Ordering Food
JSON Sends Data
APIs Are The Menu

FRONTEND

(Restaurant)



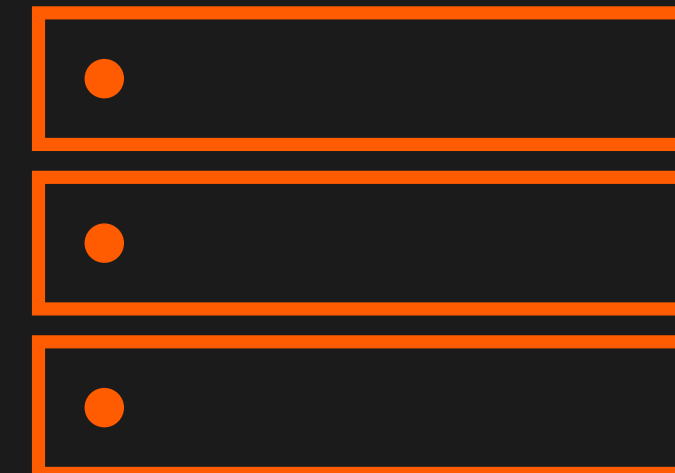
User clicks button

"I want user data"

1. HTTP REQUEST

BACKEND

(Kitchen)



2. JSON RESPONSE

`{"name": "John", "age": 25}`

API MENU:

GET /users

POST /login

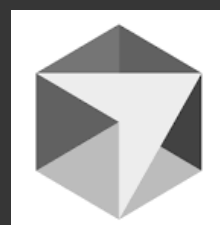
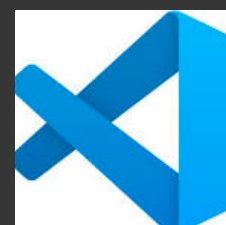
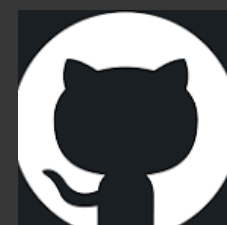
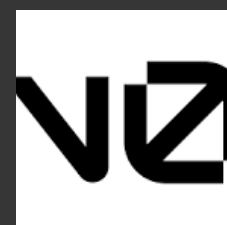
DELETE /cart



Database

Persistent storage system organizing and managing application data. Ensures data integrity, relationships, and efficient retrieval.

TOOLS OF THE TRADE



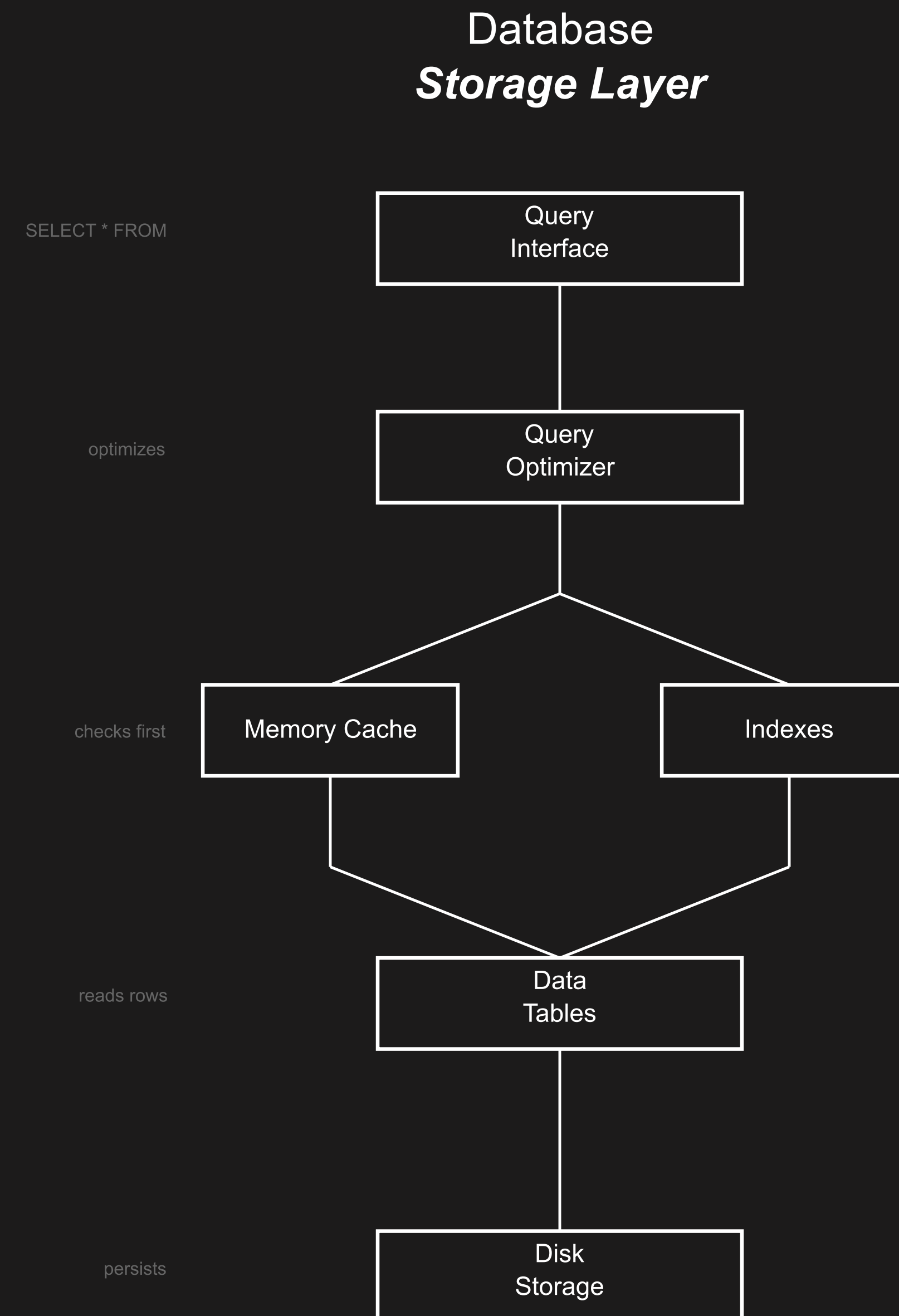
How Storage Works

YOUR APP'S MEMORY

Databases store the lifeblood of your application. They organize information into structured relationships, ensure data integrity, and retrieve exactly what you need in milliseconds. Proper indexing and caching strategies keep everything running at peak performance.

QUICK POINTS

1. Structured data storage
2. Lightning-fast queries
3. Relationship management
4. Backup & recovery

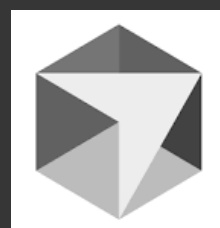
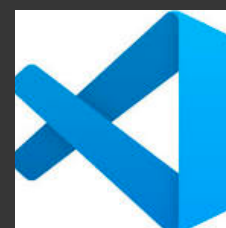
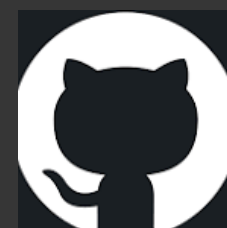
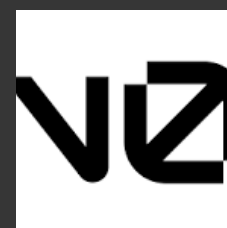




Connection

Communication protocols and APIs linking components. Manages data flow, request routing, and system integration points.

TOOLS OF THE TRADE



What Are API's?

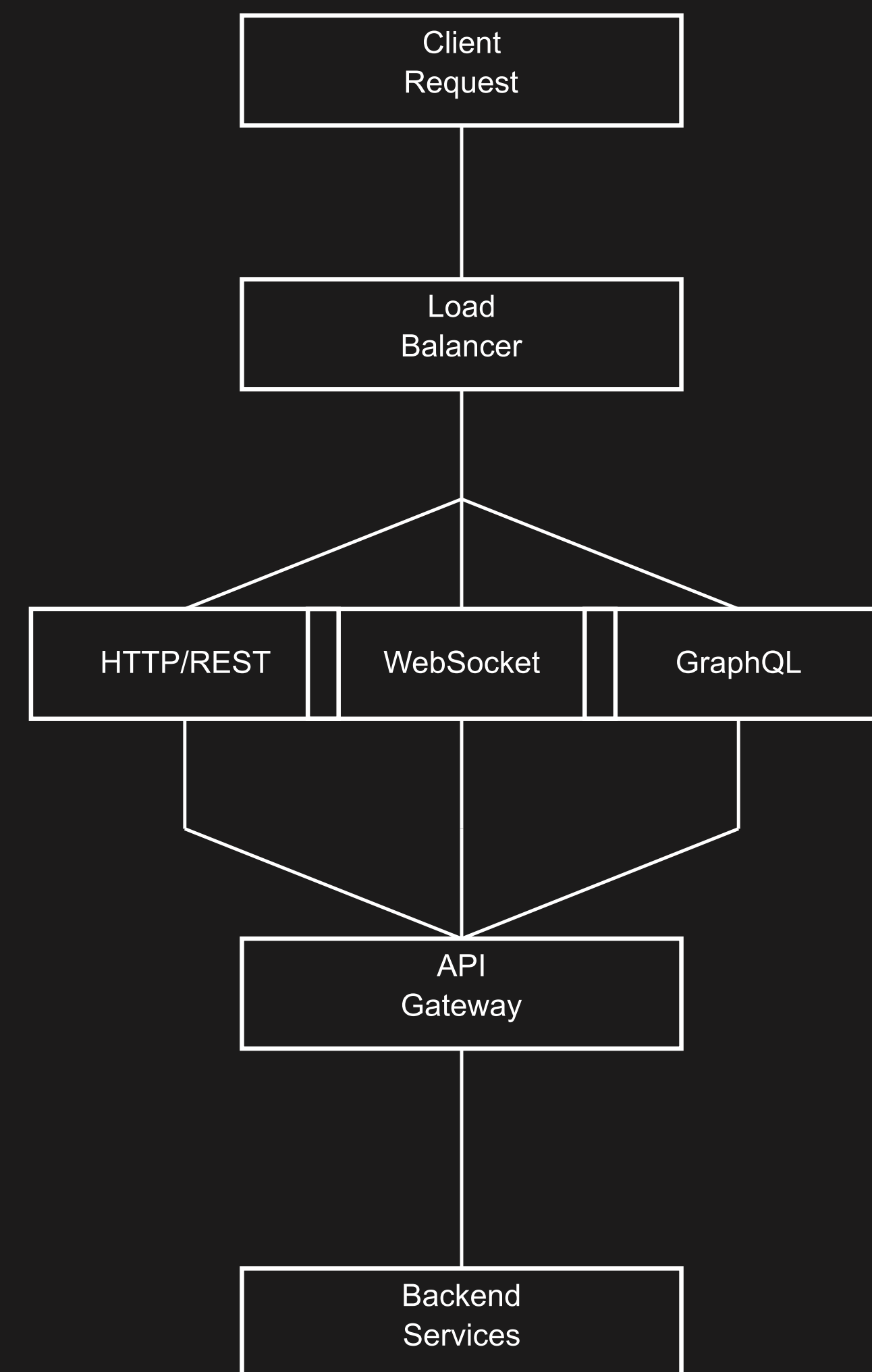
LINKING IT ALL TOGETHER

Connections are the invisible highways of your app. REST APIs exchange data between services, WebSockets enable real-time updates, and message queues handle background tasks. Every request flows through carefully orchestrated protocols.

QUICK POINTS

1. REST & GraphQL APIs
2. WebSocket streams
3. Message queuing
4. Load balancing

Connection *Architecture*





Starter Project

Personal Landing Page





Core Loops

Loop 1 – Scaffold the page (Frontend)

Loads locally, form visible, basic validation (required fields).

Loop 2 – Wire the backend (API)

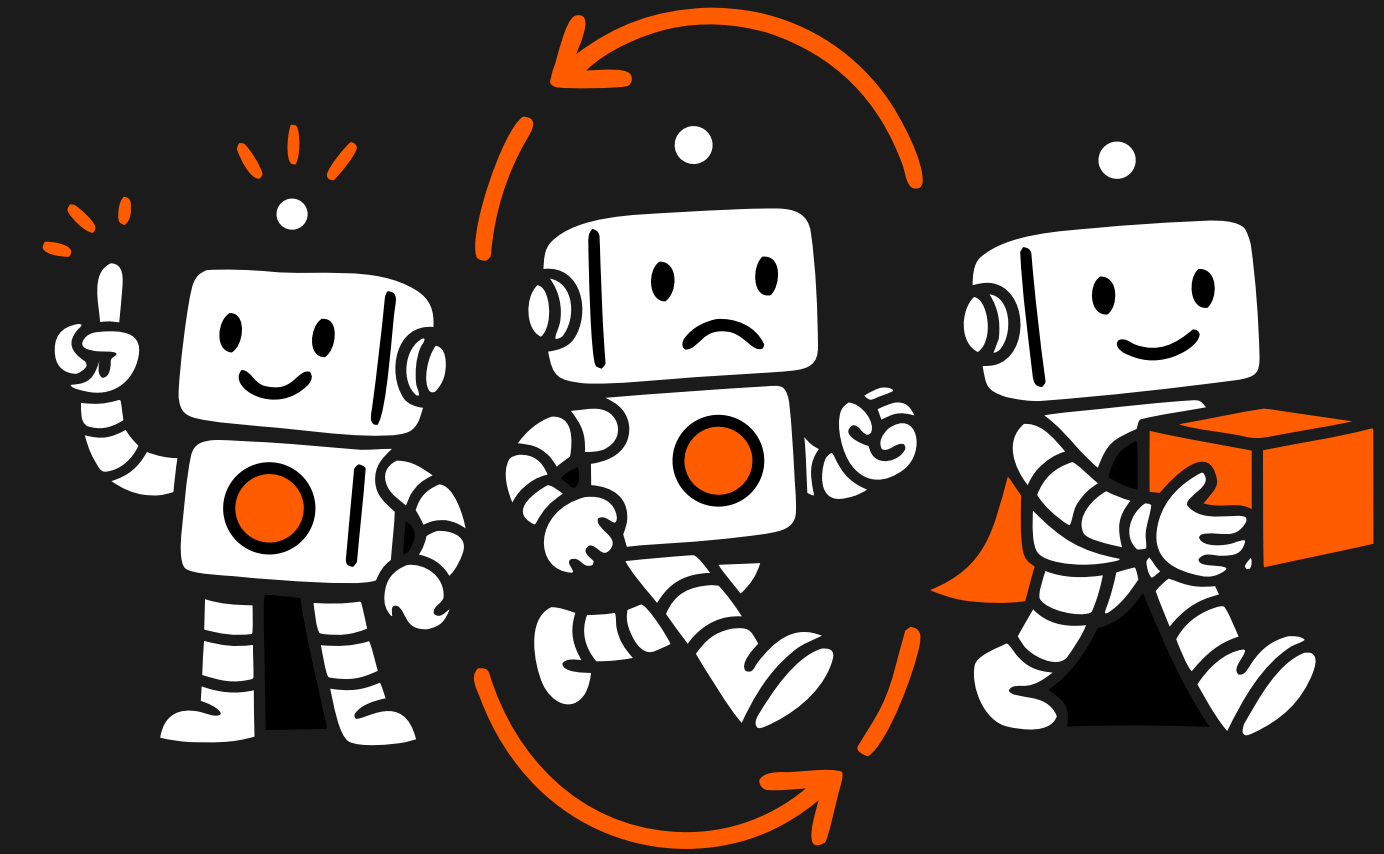
Add a POST `/api/subscribe` endpoint that accepts JSON `{name, email}`, validates email format, and returns `{ok: true}` or `{ok: false, error}` without changing the UI yet

Loop 3 – Connect to the database (Supabase option)

Integrate Supabase. Create a table `subscribers(id uuid, name text, email text unique, created_at timestamptz default now())`. Insert on `/api/subscribe`. Use env vars `SUPABASE_URL` and `SUPABASE_ANON_KEY`. Handle duplicate emails gracefully.



What This Taught Us:



Start Small

By scoping the project to a single landing page, we avoided the Doom Loop. Building one feature at a time kept progress visible and momentum high.

Guide the AI

The AI did the heavy lifting, but only because we gave it clear, structured prompts. The better the instructions, the cleaner and faster the output.

Human in the Loop

Testing after each loop caught issues early. Debugging, reviewing, and asking “why” turned AI from a black box into a teaching tool.

Ship It Live

The real learning comes when an app is deployed. Moving from local prototype → hosted site showed how all the Lego blocks (UI, backend, DB, deploy) connect in the real world.



Intermission

Quick Lunch Break